

National University of Technology

Machine Learning Lab- CS3504

LAB Submitted to:

Lec.Hajra Ahmad

Submitted By:

Muhammad Abdullah

Reg. No: **F22607010**
Batch: **Fall AI 2022**
Lab : **07**

Q.NO.1:

Write a Python Code to implement the neural network in Figure 7.1. Calculate Errors and compare

the results with the above calculations. Also, write a conclusion on the following,

1. **How the input data passes through the network, and activations are computed at each layer?**
2. **Impact of the sigmoid activation function on introducing non-linearity in the hidden and output layers.**

Answer:

```
import numpy as np

# Sigmoid activation function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Derivative of sigmoid
def sigmoid_derivative(x):
    return x * (1 - x)

# Assuming 2 input neurons, 2 hidden neurons, and 1 output neuron
np.random.seed(42)
weights_input_hidden = np.array([[0.5, 0.2], [-0.3, 0.8]])
weights_hidden_output = np.array([[0.6], [-0.4]])
bias_hidden = np.array([0.1, -0.1])
bias_output = np.array([0.05])

# Input data and expected output
X = np.array([[0.4, 0.6]])
y = np.array([[0.7]])

# Forward pass
# Input to hidden layer
hidden_layer_input = np.dot(X, weights_input_hidden) + bias_hidden
hidden_layer_output = sigmoid(hidden_layer_input)

# Hidden to output layer
output_layer_input = np.dot(hidden_layer_output, weights_hidden_output) +
bias_output
output_layer_output = sigmoid(output_layer_input)
```

```
# Error calculation
error = y - output_layer_output

# Backward pass (manual gradient calculation)
# Output layer gradients
d_output = error * sigmoid_derivative(output_layer_output)


error_hidden_layer = d_output.dot(weights_hidden_output.T)
d_hidden_layer = error_hidden_layer *
sigmoid_derivative(hidden_layer_output)

print("Forward Pass Results:")
print(f"Hidden Layer Input: {hidden_layer_input}")
print(f"Hidden Layer Output: {hidden_layer_output}")
print(f"Output Layer Input: {output_layer_input}")
print(f"Output Layer Output: {output_layer_output}")
print(f"Error: {error}")

print("\nBackward Pass Results:")
print(f"Output Gradient (d_output): {d_output}")
print(f"Hidden Layer Gradient (d_hidden_layer): {d_hidden_layer}")
```

Output:

```
Forward Pass Results:
Hidden Layer Input: [[0.12 0.46]]
Hidden Layer Output: [[0.52996405 0.61301418]]
Output Layer Input: [[0.12277276]]
Output Layer Output: [[0.53065469]]
Error: [[0.16934531]]

Backward Pass Results:
Output Gradient (d_output): [[0.04217719]]
Hidden Layer Gradient (d_hidden_layer): [[ 0.00630386 -0.00400224]]
```